Veri Yapıları Lab 6

Şube 1

Örnek 1 · Yığınla ters çevirme

```
#include <stdio.h>
#include <string.h>
#define maks 100

char stack[maks];
int top = -1;            // -1: yığın boş

void push(char x) {
    if (top == maks - 1)
        printf("stack overflow");
    else
        stack[++top] = x;
}

void pop() {
    if (top == -1)
        printf("stack underflow");
    else
        printf("%c", stack[top--]);
}
```

top = -1
Yığın boş. İlk push'ta top 0 olur.

stack[++top] = x
Önce top artar, sonra eleman yazılır.

stack[top--]
Önce eleman okunur, sonra top azalır.

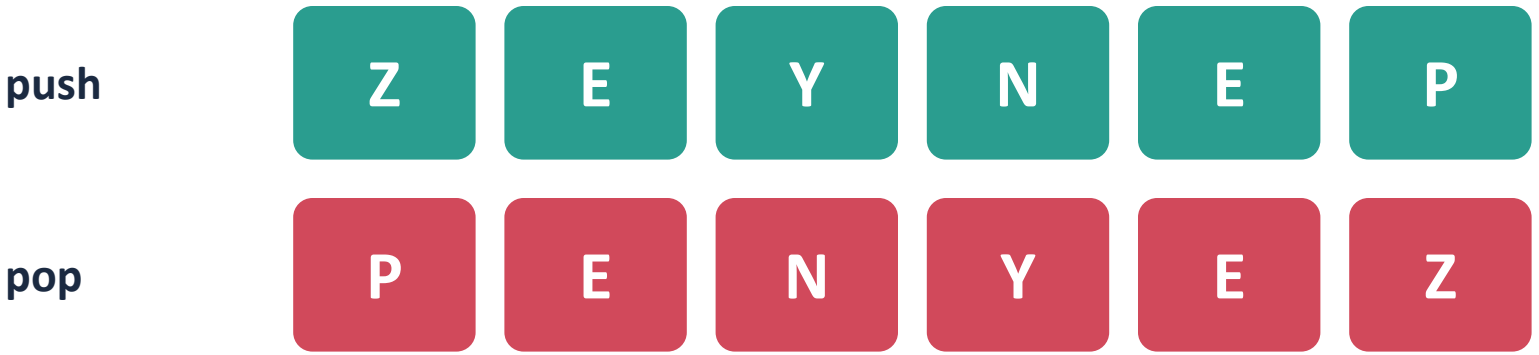
pop() yazdırır
Çıkarılan karakter ekrana basılır.

Örnek 1 · main ve akış

```
int main() {
    char ad[] = "ZEYNEP";
    int uzunluk = strlen(ad);
    for (int i = 0; i < uzunluk; i++)
        push(ad[i]);      // Z E Y N E P sırayla eklenir
    for (int i = 0; i < uzunluk; i++)
        pop();            // P E N Y E Z sırayla çıkar
    printf("\n");
    return 0;
}
```

Çıktı: PENYEZ

Eklenen sıranın tersi çıkar. Yığının LIFO özelliği bu yüzden bir ters çevirme aracıdır.



Örnek 2 · struct ile yığın

```
#include <stdio.h>
#define boyut 10

struct stack {
    int top;              // eleman sayısı = sıradaki boş indis
    int deger[boyut];
};
struct stack s;           // global olduğu için top = 0

void push(struct stack *ps, int x) {
    if (ps->top == boyut)
        printf("stack dolu\n");
    else
        ps->deger[ps->top++] = x;
}

void pop(struct stack *ps) {
    if (ps->top == 0)
        printf("stack bos\n");
    else
        ps->top--;
}
```

Örnek 2 · listeleme

```
void listele(struct stack *ps) {
    if (ps->top == 0) {
        printf("Liste bos\n");
        return;
    }
    printf("Listeniz: ");
    for (int a = 0; a < ps->top; a++)
        printf("%d ", ps->deger[a]);
    printf("\n");
}
```

Örnek 2 · menü ve deneme

```
int main() {
    int secim = 0, sayi;
    while (secim != 4) {
        printf("\n1 Ekle 2 Sil 3 Listele 4 Cikis\n");
        scanf("%d", &secim);
        switch (secim) {
        case 1:
            printf("Sayi: ");
            scanf("%d", &sayi);
            push(&s, sayi);
            break;
        case 2: pop(&s);     break;
        case 3: listele(&s); break;
        }
    }
    return 0;
}
```

Deneyelim

Giriş	Sonuç
1, 10	10 eklendi
1, 20	20 eklendi
3	Listeniz: 10 20
2	20 çıkarıldı
3	Listeniz: 10
2, 2	Liste boşalır; ikincisi: stack bos

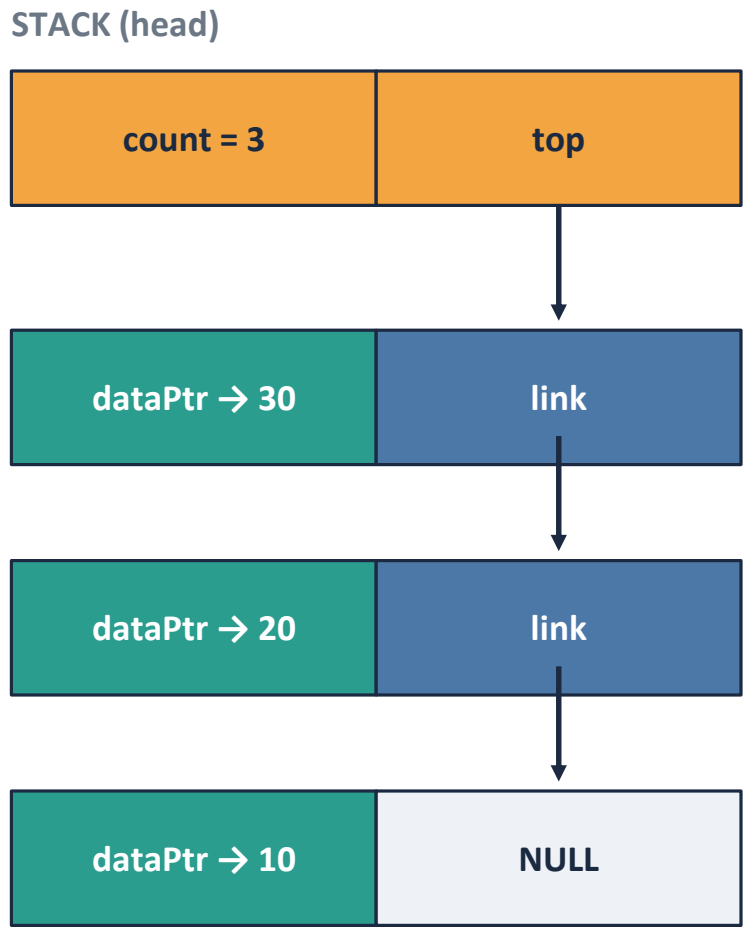
Hazırlık · bağlı liste ile yığın

```
#include <stdlib.h>
#include <stdbool.h>

typedef struct node {
    void*        dataPtr;
    struct node* link;
} STACK_NODE;

typedef struct {
    int          count;
    STACK_NODE* top;
} STACK;

bool pushStack(STACK* stack, void* dataInPtr) {
    STACK_NODE* newPtr = (STACK_NODE*)malloc(sizeof(STACK_NODE));
    if (!newPtr) return false;      // overflow
    newPtr->dataPtr = dataInPtr;
    newPtr->link    = stack->top;
    stack->top      = newPtr;
    stack->count++;
    return true;
}
```



Alıştırma Soruları

1

Palindrom kontrolü (dizi ile yığın)
Kullanıcıdan bir kelime alan, harflerini yığına ekleyip tek tek çıkararak kelimenin palindrom olup olmadığını yazdıran program yazınız. (KAYAK → Palindrom, MASA → Palindrom değil)

2

Tepeyi göster (struct ile yığın)
Örnek 2'deki menüye “Tepeyi göster” ve “Eleman sayısı” seçeneklerini ekleyiniz. Tepe eleman yığından silinmeden yazdırılmalı, yığın boşsa uyarı verilmelidir.

3

Bağlı liste ile yığın
createStack, pushStack ve popStack fonksiyonlarını yazarak yığına dinamik bellekle 3 tamsayı (5, 15, 25) ekleyiniz. Ardından hepsini pop edip ekrana yazdırınız (void * ve -> operatörü, her düğümü free ediniz).